



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0501 – DATABASE MANAGEMENT SYSTEM

TECHNICAL PRESENTATION

PRESENTED BY:

D. Gnana Deepthi (192411123)

1. DATABASE MANAGEMENT SYSTEM

- A Database Management System (DBMS) is software that creates, organizes, stores, retrieves, and manages data in a database.
- It acts as an intermediary between end users/applications and the physical database, hiding low-level storage details.
- Formally: DBMS = Database + set of programs to access and manage that data (define, construct, manipulate, share).
- Examples: Oracle, MySQL, PostgreSQL, Microsoft SQL Server, MongoDB.

OBJECTIVES OF A DBMS

- Data Abstraction & Independence — hide implementation details from users.
- Data Integrity — enforce rules/constraints so stored data remains accurate and consistent.
- Data Security — restrict unauthorized access through authentication and permissions.
- Concurrent Access — allow multiple users to access data simultaneously without conflict.
- Backup & Recovery — protect data against system or hardware failure.
- Minimizing Redundancy — avoid unnecessary duplication of data across files.

ADVANTAGES OF A DBMS

- Reduced Data Redundancy — centralized storage avoids duplicate copies of the same data.
- Data Consistency — updates are reflected uniformly across the system.
- Improved Data Security — role-based access control and authentication.
- Efficient Data Access — query languages (SQL) allow fast, flexible retrieval.
- Data Integrity — constraints (primary key, foreign key, checks) maintain correctness.
- Concurrent Access Control — transactions managed safely for many simultaneous users.
- Backup & Recovery Mechanisms — protects against data loss.

REAL-WORLD APPLICATIONS OF DBMS

- **Banking**

Account management, transactions, loan processing, fraud detection.

- **Healthcare**

Electronic Health Records (EHR), patient history, billing, lab systems.

- **Airlines & Railways**

Reservation systems, schedules, ticketing.

- **E-Commerce**

Product catalogs, orders, inventory, customer data.

- **Education**

Student records, admissions, examination results.

2. File-Based Systems vs DBMS

Aspect	File-Based System	DBMS
Data Redundancy	High — same data often duplicated across files	Minimized through centralization and normalization
Data Integrity	Difficult to enforce; scattered validation logic	Enforced centrally via constraints and rules
Data Security	Limited; file-level OS permissions only	Fine-grained, role-based access control
Concurrent Access	Prone to conflicts and data corruption	Managed via transactions and locking
Data Independence	Low — programs tightly coupled to file structure	High — logical and physical independence
Backup & Recovery	Manual, error-prone	Automated, built-in recovery mechanisms
Querying	Requires custom program for each query	Declarative query language (SQL)

CASE STUDY: BANKING — FILE-BASED VS DBMS

- **Legacy file-based branch banking (pre-1980s)**

- Each branch kept separate customer ledgers/files; no real-time link between branches.
- A customer could not deposit or check balance from a different branch ("branch banking", not "any-branch banking").
- Reconciliation between branches was manual and slow, increasing the risk of errors and fraud.

- **Modern DBMS-based Core Banking Systems (e.g. Finacle, Flexcube)**

- Centralized database lets any branch or ATM access the same live account record — enabling "anywhere banking".
- ACID transactions guarantee a fund transfer either completes fully or not at all, even during failures.
- Real-time fraud detection and audit trails are possible because all transactions sit in one consistent database.

Case Study: Healthcare — File-Based vs DBMS

- **Legacy paper/file-based patient records**

- Each hospital department (lab, pharmacy, billing) kept isolated paper or local files.
- A specialist in one department often could not see a patient's history from another department, risking duplicate tests or conflicting prescriptions.
- Records could be lost, damaged, or take a long time to retrieve in emergencies.

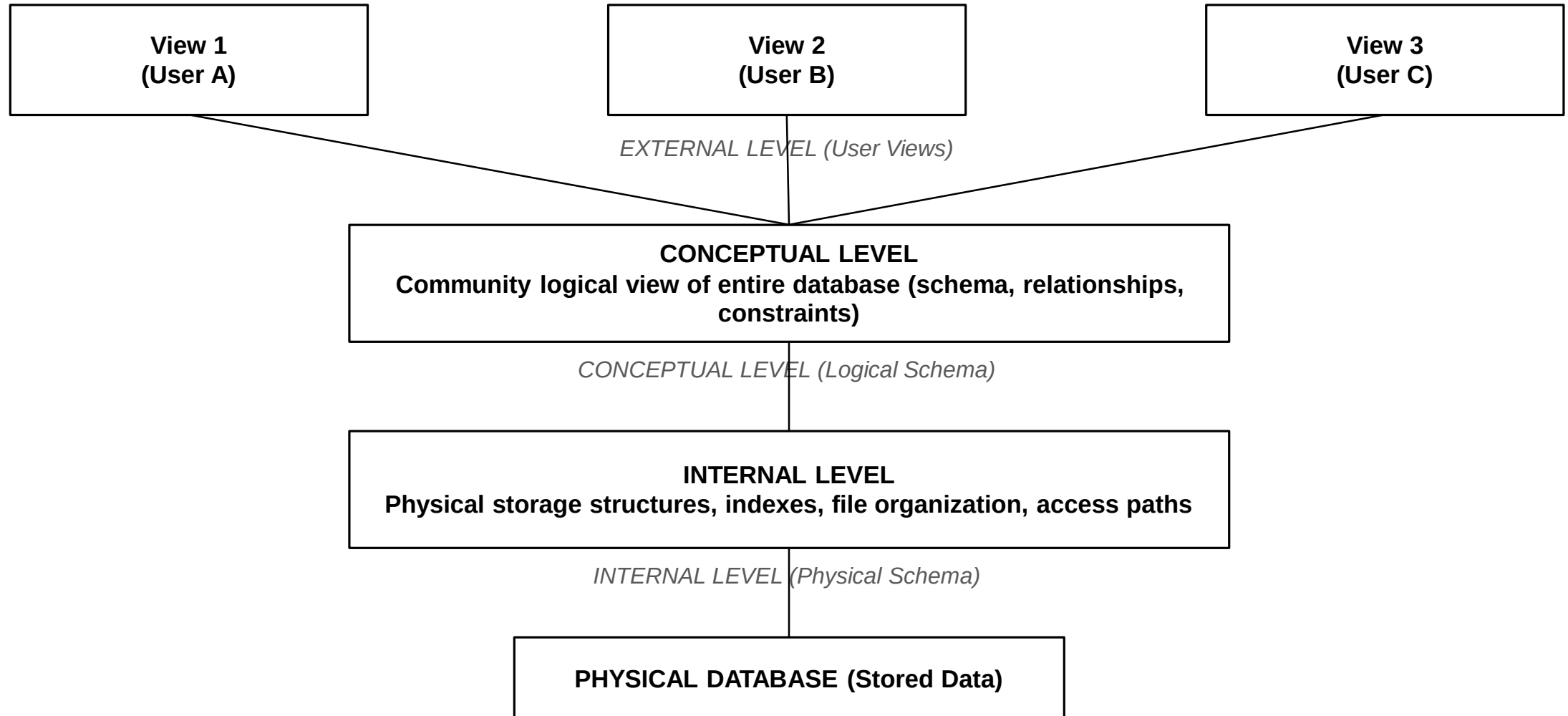
- **Modern DBMS-based Electronic Health Record (EHR) systems**

- A single patient record is shared securely across departments and even across hospitals in the same network.
- Access control ensures only authorized staff view sensitive data, satisfying regulations such as HIPAA.
- Instant retrieval of history, allergies, and medication improves emergency response and reduces medical errors.

3. THREE-LEVEL ANSI/SPARC ARCHITECTURE

- Proposed by ANSI/SPARC (1975) to separate user views from physical storage.
- Goal: provide data abstraction and data independence.
- Splits the system into three levels: External, Conceptual, and Internal.
- Each level describes the database at a different degree of abstraction.

Three-Level ANSI/SPARC Architecture — Diagram



Levels Explained with a Practical Example (University Database)

- **External Level**

- Registrar's view: student name, roll no., courses enrolled.
- Accounts office view: student name, fee status, scholarship details.
- Same underlying data, two different tailored views for two user groups.

- **Conceptual Level**

- One integrated schema: Student, Course, Enrollment, Fee tables with all relationships and constraints defined once.

- **Internal Level**

- How the Student table is actually stored: B+ tree index on roll number, file blocks on disk, storage in a specific format.

MAPPINGS BETWEEN LEVELS

- **External / Conceptual Mapping**

- Connects each external view to the conceptual schema; if the conceptual schema changes, only this mapping is updated — user views stay unaffected.

- **Conceptual / Internal Mapping**

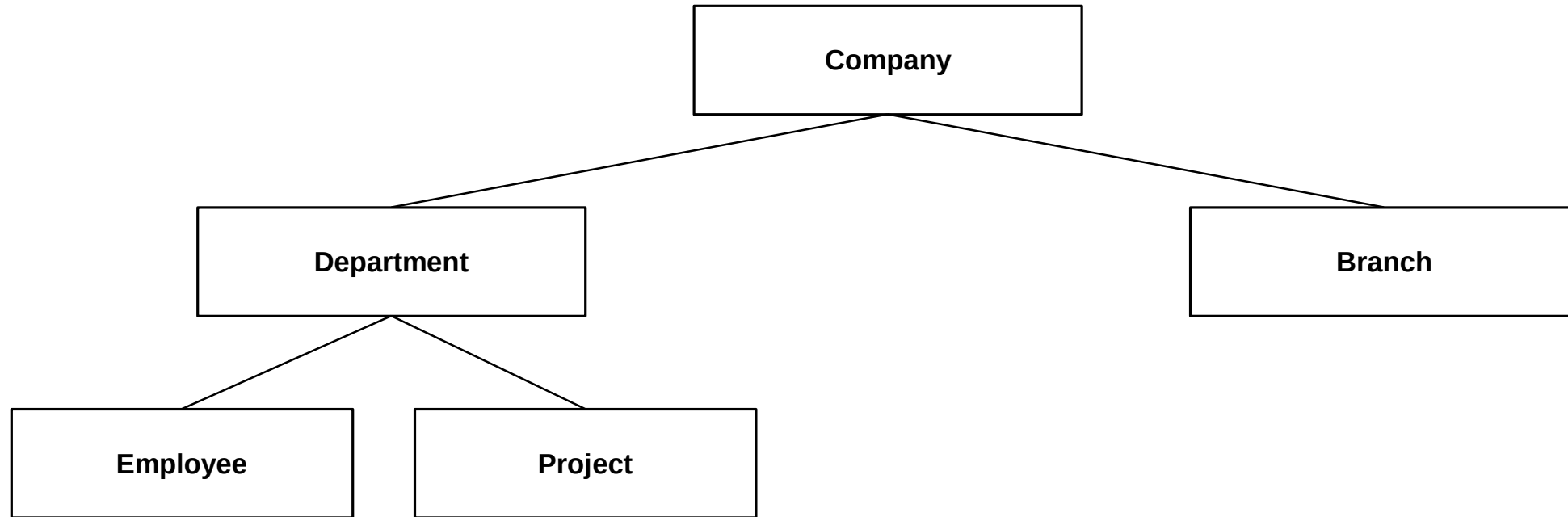
- Connects the conceptual schema to the internal (physical) schema; allows physical storage to change without altering the logical structure.

- **These mappings are exactly what make logical and physical data independence possible.**

4. DATA MODEL

- A data model defines how data is logically structured, related, and manipulated within a DBMS.
- It provides the conceptual tools to describe data, data relationships, semantics, and constraints.
- Three classical models: Hierarchical, Network, and Relational (followed later by Object-Oriented and NoSQL models).

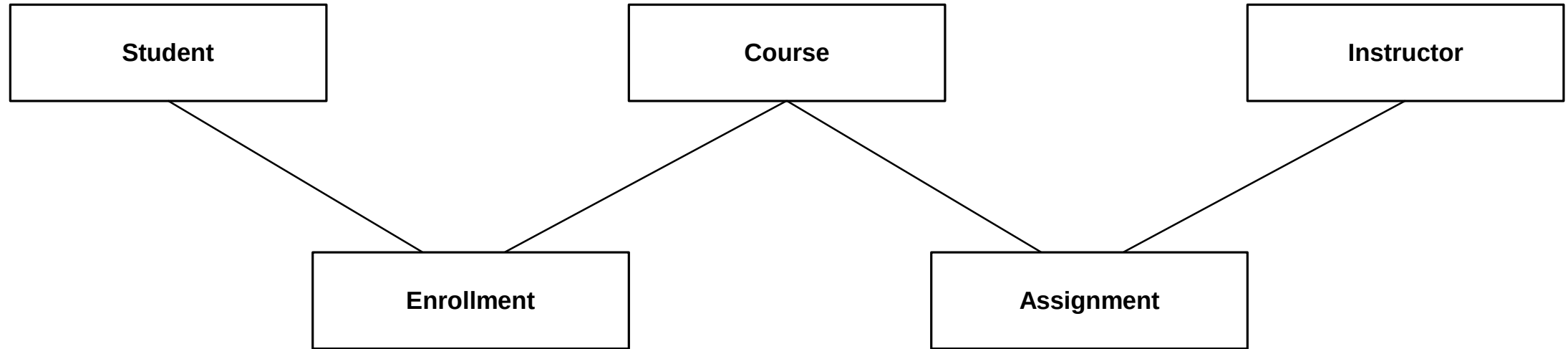
HIERARCHICAL DATA MODEL



Records are organized as a tree — each child has exactly one parent (one-to-many relationships only).

Industry Use Case: IBM's Information Management System (IMS) — still used in banking and airline mainframe systems for structured, high-volume transaction processing.

NETWORK DATA MODEL



Records can have multiple parents and multiple children — modeled as a graph, not strictly a tree (many-to-many relationships).

Industry Use Case: Integrated Data Store (IDS) and CODASYL-based systems historically used in early airline reservation and manufacturing inventory systems requiring complex interlinked records.

RELATIONAL DATA MODEL



Data is organized into two-dimensional tables (relations) of rows and columns; relationships are expressed using foreign keys — no navigational pointers needed.

Industry Use Case: Almost all modern business applications — banking (Oracle, DB2), e-commerce (MySQL, PostgreSQL), ERP systems (SAP HANA) — rely on the relational model for its simplicity and strong theoretical foundation (relational algebra, SQL).

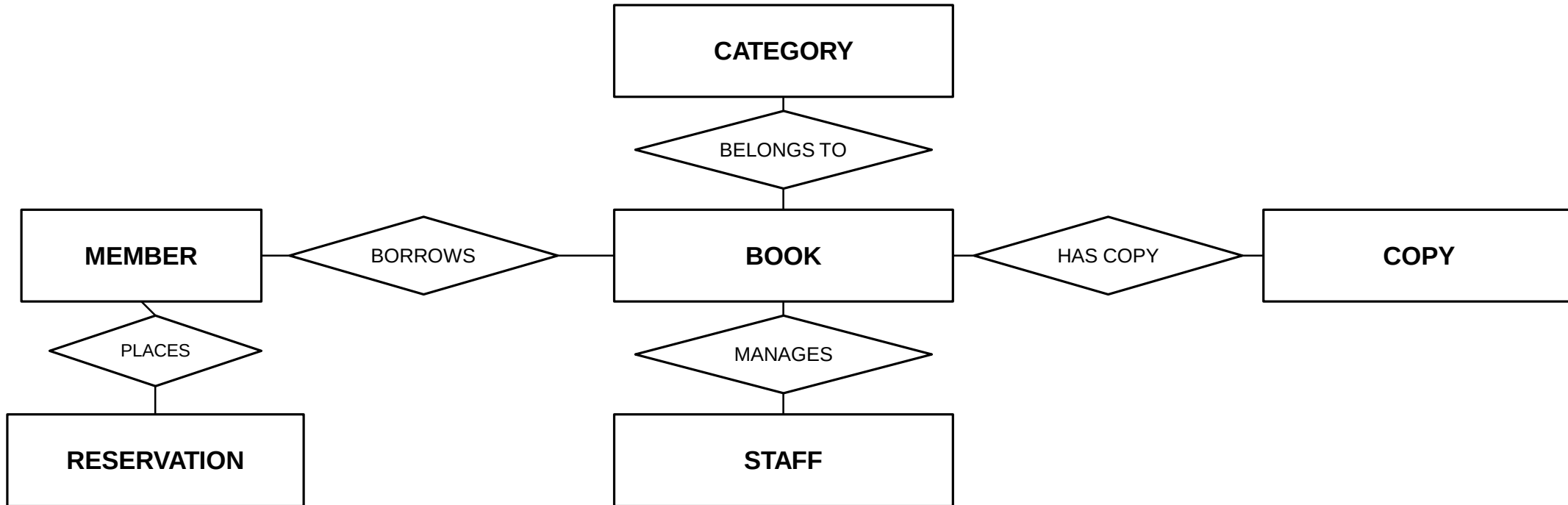
COMPARING THE THREE DATA MODELS

Aspect	Hierarchical	Network	Relational
Structure	Tree (parent-child)	Graph (owner-member)	Tables (rows & columns)
Relationships	One-to-many only	Many-to-many supported	Any, via foreign keys
Navigation	Top-down traversal	Pointer-based navigation	Declarative (SQL) queries
Flexibility	Rigid, hard to restructure	Complex but flexible links	High flexibility, easy to alter schema
Example System	IBM IMS	IDMS / CODASYL	Oracle, MySQL, PostgreSQL

5. ONLINE LIBRARY SYSTEM

- Members can browse, search, and borrow Books.
- Each Book belongs to a Category and may have multiple Copies.
- A Member can issue multiple Books over time; each issue is recorded as a Loan transaction.
- The library employs Staff who manage the catalog and process returns/fines.
- The system tracks Reservations for books currently on loan.

ER DIAGRAM — ONLINE LIBRARY SYSTEM

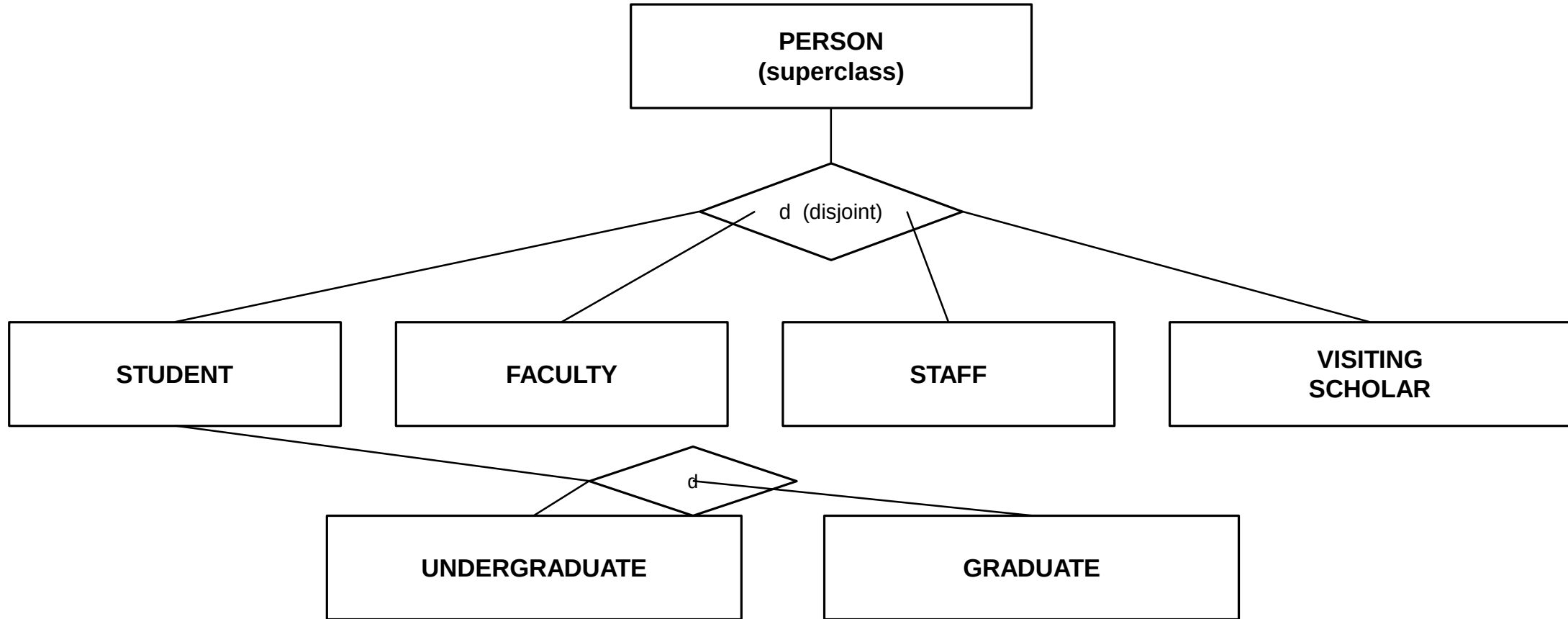


Rectangle = entity · Diamond = relationship. A Reservation records which Book a Member has requested.

6. THE ENHANCED ER (EER) MODEL

- Extends the basic ER model with concepts needed for more complex, real-world databases.
- Adds: Generalization, Specialization, Categorization (Union), and Attribute/Relationship inheritance.
- Generalization — bottom-up process: combine lower-level entities that share common attributes into a higher-level (super-class) entity.
- Specialization — top-down process: divide a higher-level entity into lower-level (sub-class) entities based on distinguishing characteristics.

University Case Study — Generalization / Specialization



PERSON specializes into STUDENT / FACULTY / STAFF / VISITING SCHOLAR (disjoint, based on "Role"). STUDENT further specializes into UNDERGRADUATE / GRADUATE.

7. LOGICAL DATA INDEPENDENCE

- The capacity to change the conceptual (logical) schema without requiring changes to external schemas or application programs.
- Example: Adding a new attribute (e.g. Email) to the Student table, or adding a new entity, does not force existing application queries to be rewritten.
- Achieved through the External / Conceptual mapping in the ANSI/SPARC architecture.
- Generally harder to achieve than physical independence, since logical changes can affect the meaning of data seen by users.

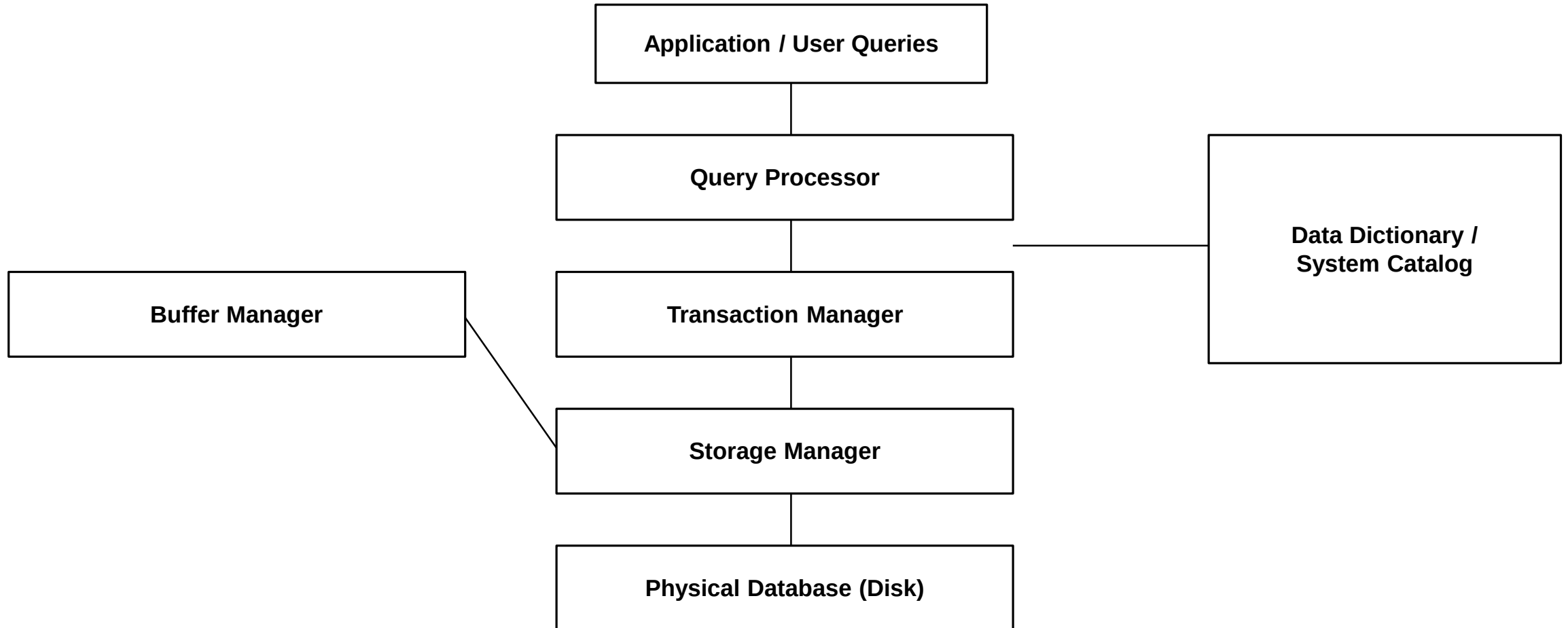
PHYSICAL DATA INDEPENDENCE

- The capacity to change the internal (physical) schema — storage structures, file organization, indexing — without altering the conceptual schema or application programs.
- Example: Switching a table's storage from a heap file to a B+ tree indexed file, or moving data to new disk hardware, is invisible to applications.
- Achieved through the Conceptual / Internal mapping in the ANSI/SPARC architecture.
- Easier to achieve than logical independence, because it does not affect the logical meaning of the data — only how it is stored.

SIGNIFICANCE OF DATA INDEPENDENCE IN DBMS ARCHITECTURE

- Reduces maintenance cost — schema or storage changes do not ripple through every application.
- Supports evolution — databases can grow and be optimized over time without breaking existing software.
- Enables true separation of concerns across the three ANSI/SPARC levels (external, conceptual, internal).
- Foundational goal that distinguishes a modern DBMS from simple file-processing systems.

DBMS Software Architecture — Component Overview



QUERY PROCESSOR

- Translates high-level queries (SQL) into low-level instructions the storage manager can execute.
- **Key sub-components:**

DDL Interpreter — processes schema definitions (CREATE, ALTER).

DML Compiler — translates DML statements into an execution plan (relational algebra expression).

Query Optimizer — chooses the most efficient execution strategy among alternatives.

Query Evaluation Engine — executes the chosen plan and returns results.

STORAGE MANAGER

- Bridges the gap between the application/query layer and the physical operating-system files.
- **Key sub-components:**

Authorization & Integrity Manager — checks access permissions and integrity constraints.

File Manager — manages allocation of disk space and the file structures used to store data.

Buffer Manager — decides which data to cache in main memory for fast access.

Ensures data is read from and written to disk efficiently and correctly.

TRANSACTION MANAGER

- Ensures the database remains in a consistent state even with concurrent access and system failures.
- **Responsible for the ACID properties:**

Atomicity — a transaction executes completely or not at all.

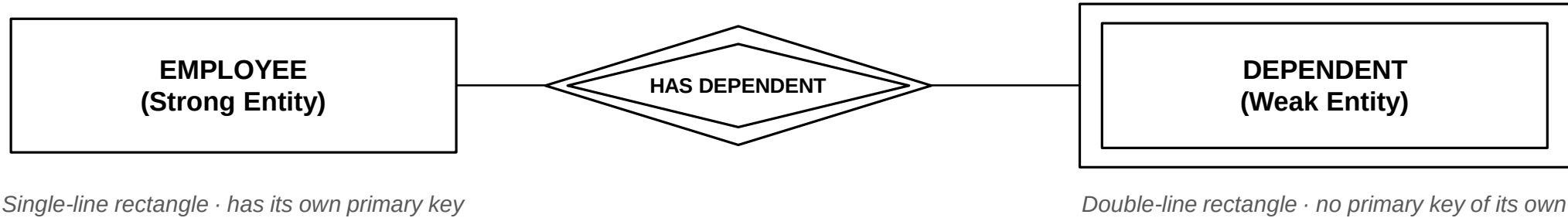
Consistency — a transaction moves the database from one valid state to another.

Isolation — concurrent transactions do not interfere with each other's intermediate results.

Durability — once committed, changes persist even after a crash.

Includes the Concurrency-Control Manager and the Recovery Manager.

9. STRONG ENTITY VS WEAK ENTITY — ER NOTATION



The double diamond is an Identifying Relationship — EMPLOYEE (owner) is required for DEPENDENT to exist.

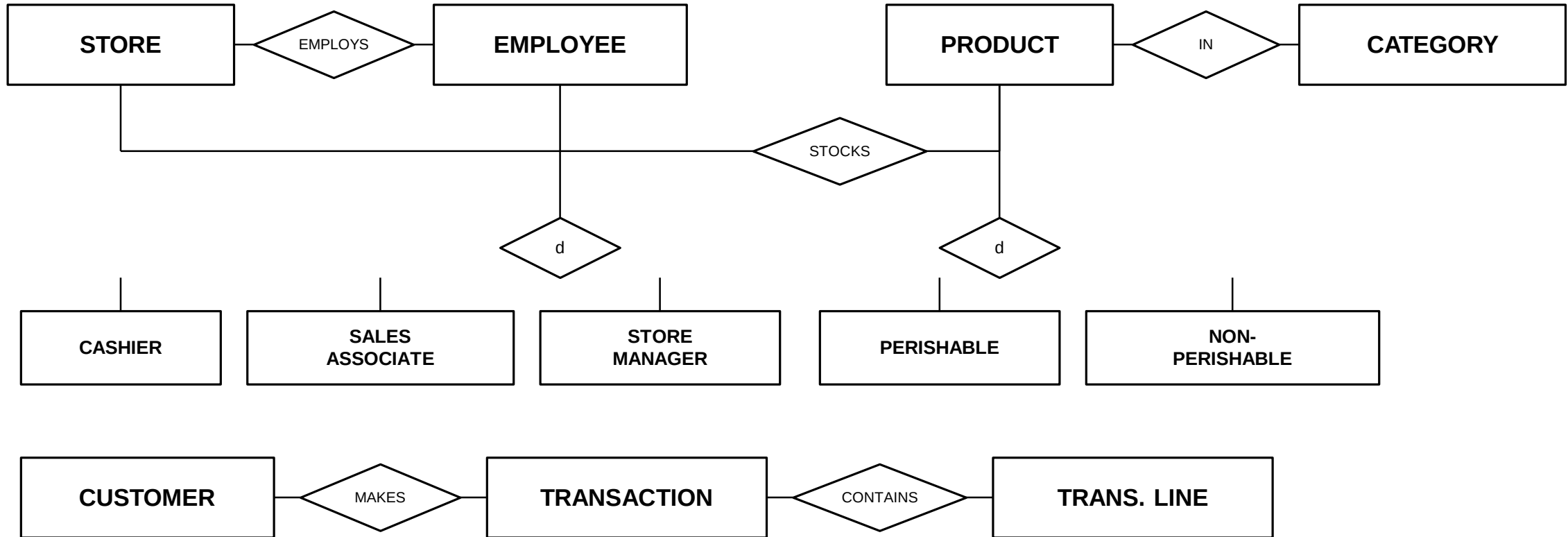
Strong vs Weak Entities — Comparison & Real-World Examples

Aspect	Strong Entity	Weak Entity
Primary Key	Has its own unique primary key	No primary key of its own — uses a partial key + owner's key
Existence	Exists independently	Existence depends on a related (owner) strong entity
ER Notation	Single-line rectangle, single diamond	Double-line rectangle, double diamond (identifying relationship)
Example 1	EMPLOYEE (Emp_ID as PK)	DEPENDENT (depends on Employee, uses Dependent_Name + Emp_ID)
Example 2	ORDER (Order_ID as PK) in e-commerce	ORDER_ITEM (identified only via Order_ID + line number)
Example 3	HOTEL (Hotel_ID as PK)	ROOM (Room_No is unique only within a specific hotel)

10. Retail Chain Management System — Requirements

- A retail company operates multiple Stores across different cities, each with its own Manager.
- Each Store sells Products organized under Product Categories, and maintains its own Inventory.
- Customers make Purchases; each purchase is a Transaction containing one or more Products.
- Employees are specialized into Cashiers, Sales Associates, and Store Managers, each with distinct attributes.
- Products are specialized into Perishable Goods (with an expiry date) and Non-Perishable Goods (with a shelf life category).

EER Diagram — Retail Chain Management System



d = disjoint specialization. Each subclass inherits the attributes of its superclass and adds its own.

Retail Chain EER — Design Walkthrough Explained

- **Core Entities**

STORE, EMPLOYEE, PRODUCT, CATEGORY, INVENTORY, CUSTOMER, TRANSACTION, and TRANS. LINE (a weak entity identified by Transaction + line number).

- **Key Relationships**

EMPLOYS (Store–Employee), STOCKS (Store–Product via Inventory), IN (Product–Category), MAKES (Customer–Transaction), CONTAINS (Transaction–Trans. Line).

- **Specialization: Employee**

EMPLOYEE is specialized (disjoint) into CASHIER, SALES ASSOCIATE, and STORE MANAGER — each inherits Name/ID/Store from Employee and adds its own attributes (e.g. Manager adds Authorization_Level).

- **Specialization: Product**

PRODUCT is specialized (disjoint) into PERISHABLE (adds Expiry_Date) and NON-PERISHABLE (adds Shelf_Life_Category), supporting different inventory rules per type.

- **Why EER Fits Here**

Plain ER cannot cleanly express that different employee/product types carry different attributes — generalization/specialization captures this naturally while avoiding repeated attribute definitions.